DESIGN (Kotlin Edition)

Repository: TC53eBSP_TestRFD40

This document defines the current RFID connection lifecycle and event behavior for: - init - connect - disconnect - reader detach/attach - power cable connected/disconnected

It is aligned with: - `app/src/main/java/com/zebra/rfid/demo/sdksample/MainActivity.kt` - `app/src/main/java/com/zebra/rfid/demo/sdksample/RFIDHandler.kt`

1) Architecture

- `MainActivity`
  - Owns Android lifecycle and permission flow.
  - Owns USB power broadcast receiver with active handling for `ACTION_POWER_DISCONNECTED`.
  - Maintains reconnect-window state (`powerReconnectWindowActive`) and UI suppression state (`suppressReconnectStatusUntilConnected`).
  - Calls `RFIDHandler` lifecycle methods.
- `RFIDHandler`
  - Owns SDK init/discovery/connect/disconnect and reconnect guards.
  - Owns reader attach/disappear callbacks via `Readers.RFIDReaderEventHandler`.
- `RFIDHandler.EventHandler`
  - Owns runtime reader status events (`DISCONNECTION_EVENT`, trigger events, read events).

2) State and Concurrency Guards

`RFIDHandler` uses:

```
@Volatile
private var initializationInProgress = false

@Volatile
private var connectionInProgress = false

@Volatile
private var resumeRequested = false

@Volatile
private var readersAttached = false

@Volatile
private var detachedEventInProgress = false

@Volatile
private var skipTc53eReaderSelection = false
```

Guard intent: - `resumeRequested`: foreground intent to stay connected. - `initializationInProgress`: prevents duplicate `initSdk()`. - `connectionInProgress`: prevents connect overlap. - `readersAttached`: prevents duplicate `Readers.attach(this)`. - `detachedEventInProgress`: suppresses redundant “Disconnected” toast when detach already reported.

`MainActivity` adds reconnect UX and bounded retry guards:

```
private const val READER_RESUME_DEBOUNCE_MS = 1200L
private const val POWER_CONNECTED_EVENT_DEBOUNCE_MS = 2000L
private val POWER_RECONNECT_ATTEMPT_DELAYS_MS = longArrayOf(500L, 1200L, 2500L)
private const val POWER_RECONNECT_SUPPRESSION_TIMEOUT_MS = 11000L
private const val POWER_RECONNECT_BUSY_RETRY_DELAY_MS = 400L

private var suppressReconnectStatusUntilConnected = false
private var powerReconnectWindowActive = false
private var powerReconnectAttemptIndex = 0
private var powerReconnectStartedAtMs = 0L
```

Intent: - debounce burst reconnect requests (`onPostResume`, USB events). - schedule staged reconnect retries after cable unplug. - retry with short delay when handler is busy (`isConnectionBusy()`). - hide noisy intermediate reconnect errors until final connected status or timeout.

3) Lifecycle Contract

3.1 Init

Entry points: - `MainActivity.initializeRfidHandlerIfPermitted()` -> `rfidHandler.onCreate(this, this)` - `RFIDHandler.onResume()` when `readers == null`

Behavior: 1. `onCreate` immediately calls `initSdk()`. 2. `initSdk()` is idempotent while `initializationInProgress == true`. 3. Background `createInstanceAndConnect()` builds `Readers(appContext, ENUM_TRANSPORT.ALL)`. 4. If reader list empty, retry discovery via `RE_USB` up to `MAX_DISCOVERY_RETRIES`. 5. If `resumeRequested` is still true, hand off to `connectReader()`.

Expected log shape: - `STEP: InitSDK` - `STEP: InitSDK initializationInProgress, createInstanceAndConnect` - duplicate calls show `STEP: Skip Duplicated InitSDK`

3.2 Connect

Entry points: - `RFIDHandler.onResume()` - `RFIDReaderAppeared(...)` - `EventHandler.DISCONNECTION_EVENT` recovery

Behavior: 1. `connectReader()` exits early when any guard blocks (`!resumeRequested`, init in progress, connect in progress, already connected). 2. `getAvailableReader()` ensures `Readers.attach(this)` once-per-cycle. 3. Reader selection prefers RFD403 family (eConnex), then RFD40+, then first fallback. 4. `connect()` measures connection duration, calls `configureReader()`, reports status. 5. `finishConnectionAttempt(...)` always clears `connectionInProgress`.

Expected log shape: - `STEP: connectReader connectionInProgress` - `STEP: getAvailableReader` - `STEP: finishConnectionAttempt(connect())` - `STEP: Reader Connected in Xms`

3.3 Disconnect

Entry points: - `MainActivity.onPause()` - menu disconnect action - USB `ACTION_POWER_CONNECTED` - `RFIDReaderDisappeared(...)` - `DISCONNECTION_EVENT` cleanup

Behavior: 1. `RFIDHandler.onPause()` sets `resumeRequested = false`, `connectionInProgress = false`, then `disconnect()`. 2. `disconnect()` removes event listener, disconnects reader, clears `reader`, resets attach flag in `finally`. 3. If detach callback already fired (`detachedEventInProgress = true`), generic disconnect toast is suppressed.

4) Reader Attach / Detach Event Handling

Attach (`RFIDReaderAppeared`)

Behavior: 1. Toast "Reader attached: ". 2. Audible appear tone. 3. Schedule `connectReader()`.

Detach (`RFIDReaderDisappeared`)

Behavior: 1. Set `detachedEventInProgress = true`. 2. Toast "Reader detached: ". 3. Call `disconnect()` for cleanup.

Design result: - detach is treated as authoritative state loss. - next connection cycle re-attaches via `Readers.attach(this)` because `readersAttached = false` is forced in `disconnect()`.

Flowchart: Attach

```
flowchart TD
    A([RFIDReaderAppeared]) --> B[Toast: Reader attached]
    B --> C[beepAppear tone]
    C --> D[scheduleOnBackground\nconnectReader]
    D --> E([connectReader\nif guards pass])
```

Flowchart: Detach

```
flowchart TD
    A([RFIDReaderDisappeared]) --> B[detachedEventInProgress = true]
    B --> C[Toast: Reader detached]
    C --> D[disconnect]
    D --> E[removeEventsListener]
    E --> F[reader.disconnect]
    F --> G[reader = null]
    G --> H[readersAttached = false\ndetachedEventInProgress = false\nin finally block]
    H --> I([generic Disconnected toast suppressed])
```

5) Power Cable Behavior (TC53e / EM45 flow)

Power handling is implemented in `MainActivity.mBatteryReceiver`.

5.1 `ACTION_POWER_CONNECTED`

The power connected behavior is primarily used to disconnect RFID sessions on USB-sharing devices (like TC53e) when a power-only cable is attached, or handle specific behaviors during file transfer modes.

5.2 ACTION_POWER_DISCONNECTED -> reconnect with suppression

Current behavior: 1. log ACTION_POWER_DISCONNECTED and clear powerConnectedLatched. 2. guard exits when rfidHandler == null. 3. call rfidHandler.setSkipTc53eReaderSelection(false) so host reader skip mode is disabled. 4. if isReaderConnected() is already true, stop reconnect window, emit toast, and return. 5. otherwise call startPowerReconnectWindow().

startPowerReconnectWindow() behavior: 1. cancel any previous reconnect window/timer state via stopPowerReconnectWindow(). 2. set window state: - powerReconnectWindowActive = true - powerReconnectAttemptIndex = 0 - powerReconnectStartedAtMs = SystemClock.elapsedRealtime() - suppressReconnectStatusUntilConnected = true 3. schedule first attempt with delay POWER_RECONNECT_ATTEMPT_DELAYS_MS[0] (500ms). 4. schedule hard timeout at POWER_RECONNECT_SUPPRESSION_TIMEOUT_MS (11s).

powerReconnectAttemptRunnable behavior: 1. abort when window inactive. 2. stop window if reader becomes connected. 3. if handler is busy (isConnectionBusy()), reschedule self after POWER_RECONNECT_BUSY_RETRY_DELAY_MS (400ms). 4. request reconnect via requestReaderResumeDebounced("power_disconnected_retry_<n>", true). 5. schedule next staged attempt at 1200ms then 2500ms until the bounded attempt list is exhausted.

Status suppression behavior while reconnecting: 1. sendToast(...) and onReaderStatusUpdate(...) call shouldSuppressReconnectStatus(...). 2. non-connected intermediate statuses are suppressed while suppressReconnectStatusUntilConnected is true. 3. first status containing "connected" logs elapsed time, stops window, and clears suppression. 4. on timeout, suppression is cleared and UI shows: "USB in used!!!\r\nUnplug the USB cable for reconnect" when still disconnected.

5.3 Verified Lifecycle Logs (USB Charging Flow)

The following log trace illustrates a successful power-plug disconnect followed by a power-unplug reconnect on a TC53e device:

1. **Cable Plugged In**:
   - ACTION_POWER_CONNECTED received.
   - RFIDReaderDisappeared fired by SDK (hardware-level USB release).
   - RFIDHandler executes disconnect().
   - STEP: ACTION_POWER_CONNECTED in file-transfer mode -> keep RFID connected until reader disappears (Log indicates intentional wait for authoritative hardware detach).
2. **Cable Unplugged**:
   - ACTION_POWER_DISCONNECTED received.
   - Starting power-unplug reconnect window (500ms delay).
3. **Automatic Reconnect**:
   - Power reconnect attempt 1/3 executes.
   - RFIDHandler onResume: connecting -> connectReader().
   - SDK detects hardware: Found RFID Readers Size = 1 -> Available reader: RFIDTC53E.
   - Successful connection: STEP: Reader Connected in 1900ms.
   - UI serial numbers fetched: Device serials - Android: TC53E_..., Reader: RFIDTC53E.

6) Event-to-Action Matrix

Event	Owner	Action	Target Outcome
App create	MainActivity	init handler and call `onCreate`	SDK init starts
App post-resume	MainActivity	`requestReaderResumeDebounced`	connect or resume existing connection
App pause	MainActivity	`rfidHandler.onPause()`	disconnect cleanly
Reader appeared	RFIDHandler	toast + tone + `connectReader`	attach/reconnect quickly
Reader disappeared	RFIDHandler	mark detached + toast + disconnect	clean state and stop I/O
SDK disconnection event	EventHandler	disconnect + reconnect if active	auto recovery while foreground
Power connected	MainActivity	disconnect RFID (if not TC22R or file transfer)	release reader share for USB
Power disconnected	MainActivity	debounced reconnect with suppression	stable reconnect without noisy UX

7) Required Handling Rules (Normative)

These rules should remain true when code is modified:

1. Init idempotency
   - duplicate `initSdk()` calls must be harmless.
2. Single in-flight connect
   - never allow parallel connect attempts.
3. Detach-first cleanup
   - detach must always clear listener + connection state.
4. Cable-in policy state
   - `ACTION_POWER_CONNECTED` handles reader release for host-shared devices.
5. Cable-out reconnect
   - on power disconnected, reconnect through bounded backoff attempts.
6. Suppressed transient errors
   - during cable-out reconnect, hide interim errors until final connected status or reconnect timeout.

8) Suggested Next Increment

Add telemetry counters for reconnect outcomes: - count attempts per unplug cycle - measure time-to-reconnect - track timeout frequency by device model

This will let reconnect constants be tuned using production evidence.